Session 20 Homework — Prove the Lock Is Per-Project, Not Global

CORE, STRETCH, and CHALLENGE are all required — every one of them must be completed. The three names describe increasing difficulty only, nothing about which parts you submit.

If you are doing this alone tonight and this is your first week really writing Prisma code without an instructor next to you: read "How to start" under CORE before writing anything. It will not give you the answer, but it will tell you exactly where to look first.

Context

By the end of class, `createIssueWithKey()` creates a real issue with an atomically-generated key, inside a real interactive `$transaction`, proven correct even under real concurrent operations on the SAME project. Nobody has asked you yet whether that same real guarantee comes at the cost of unrelated work slowing down — or to connect the Prisma code back to Block 4's own raw SQL in writing.

Goal

A real, runnable script proving that concurrent issue-creations on DIFFERENT projects stay genuinely parallel, never serialized by each other (CORE); a written, line-by-line correspondence between `createIssueWithKey()` and Block 4's own real SQL transaction (STRETCH); and a self-researched, written comparison against Prisma's raw SQL interop as an alternative locking strategy (CHALLENGE).

What you already have

A working, transaction-protected `createIssueWithKey()` in `api/src/prisma/prisma.service.ts`, already proven correct when two calls race on the SAME project (the live concurrency demo). You are not building a new mechanism tonight — you are extending the SAME proof technique to answer a question nobody tested live: what happens across two DIFFERENT projects?

What to build

- A new file, `api/prisma/homework-concurrency-proof.ts`, with its own version of `createIssueWithKey()`, modified so that lock-waiting becomes measurable.
- Measure and print: how long two concurrent creates on the SAME project take, versus how long two concurrent creates on DIFFERENT projects take.
- Two written answers (see STRETCH/CHALLENGE).

CORE (required)

Setup — a standalone script needs its own environment

`api/prisma/homework-concurrency-proof.ts` is a plain standalone script, run with `ts-node` — unlike the Nest app itself, nothing loads `api/.env` for it automatically. Before writing anything else, add this as the very first line of the file:

TS

```ts
import 'dotenv/config';
```

and install the real package it needs, once:

BASH

```bash
npm install --save-exact --save-dev dotenv@16.4.5
```

This is setup, not part of tonight's actual discovery — without it, the script fails immediately with `Environment variable not found: DATABASE_URL`, before any of the real concurrency logic even runs.

Build a measurable experiment comparing two concurrent issue creations:

- **A.** Two concurrent creates on the SAME project.
- **B.** Two concurrent creates on two DIFFERENT projects.

Keep the transaction open long enough that lock-waiting becomes measurable (the real lock itself needs no delay to exist — you are only making it visible to a stopwatch). Prove, from real timing, that the lock is scoped to the project row, not global: print both elapsed times, and state clearly (as printed output) whether each result matched the expectation (same-project: the second call's own work genuinely waits; different-project: the two stay genuinely parallel).

How to start

Don't start by writing the timing/comparison logic. Start by understanding the mechanism you're about to measure:

1. Open `api/src/prisma/prisma.service.ts` and re-read `createIssueWithKey()`.
2. Identify which exact database operation inside it is the one that acquires the real row lock.
3. Ask yourself: where, relative to that operation, would something need to happen for the lock to still be HELD while you measure it? (Think about what "measuring a lock" actually requires — the lock has to still exist at the moment you're timing.)
4. Get one modified, delayed version of the function working and callable ONCE, successfully, on its own, before you touch concurrency or timing at all. A function that isn't correct when called once cannot become correct by calling it twice concurrently.
5. Only once step 4 works: run two calls of it at the same time (not one after another), measure how long that takes, on the SAME project first — then repeat on two DIFFERENT real project ids, and compare.

Hints

- **Hint 1 (conceptual):** Ask yourself what row Postgres actually locks when the relevant statement inside the transaction runs. If two calls touch the SAME row, what has to happen before the second one's own write can proceed? If they touch two DIFFERENT rows, is there any reason for one to wait on the other at all?
- **Hint 2 (where to look):** You already saw the SAME-project case live — find that script or demo and reuse its shape for the SAME-project half of this proof. For the DIFFERENT-project half, you need two real project ids that are not the same row. A real delay is usually built from `setTimeout` wrapped in a `Promise` so it can be `await`ed inside an `async` function.
- **Hint 3 (API/tool name):** To run two calls at the same time rather than one after another, look at `Promise.all`. To measure real elapsed time around that call, look at `Date.now()` or `console.time` / `console.timeEnd`.

STRETCH (required)

Write a short, direct comparison (as a comment above `createIssueWithKey()`, or in your own short notes): for each of Block 4's own four real SQL pieces (`BEGIN`, `UPDATE ... RETURNING`, `INSERT`, `COMMIT`), name the exact Prisma code that plays the same role. Then name the one thing Prisma handles for you automatically that raw SQL requires you to write by hand (think about what happens on the error path).

How to start

Open Session 18's own raw-SQL transaction (the real `BEGIN` / `UPDATE ... RETURNING` / `INSERT` / `COMMIT` sequence) side by side with today's `createIssueWithKey()`. Go through the four SQL pieces one at a time, in order, and ask: "which exact line of today's Prisma code does the same real job as this line?" — write that pairing down before worrying about the "automatic" question.

Hints

- **Hint 1 (conceptual):** One of the four SQL pieces doesn't have an obvious one-line Prisma equivalent anywhere in the code — that absence is itself the answer to "what does Prisma handle automatically."
- **Hint 2 (where to look):** Think about what you would have to write BY HAND in raw SQL the moment something fails halfway through the sequence, versus what currently happens in the Prisma version when `createIssueWithKey()` throws.

CHALLENGE (required)

Research Prisma's own `$queryRaw` / `$executeRaw` (tagged-template raw SQL functions).

What to research

1. What is `$queryRaw` / `$executeRaw`, concretely — what does it let you send to the database that Prisma's normal model methods (`.update()`, `.create()`, ...) do not?
2. Can `$queryRaw` / `$executeRaw` be used INSIDE an interactive `$transaction` callback, alongside normal `tx.model.method()` calls, or are they mutually exclusive?

3. What real problem does `SELECT ... FOR UPDATE` solve, in general — and is it a problem `createIssueWithKey()` actually has?
4. Why did today's live-coded version NOT need `SELECT ... FOR UPDATE`?

Then write short, direct answers:

1. Sketch (in SQL or pseudocode, not required to run) how you would express Block 4's own `SELECT ... FOR UPDATE` pattern through `tx.$queryRaw` / `tx.$executeRaw`, instead of relying on the plain `UPDATE` statement's own implicit lock.
2. Is this explicit version more correct than this session's live-coded version, less correct, or equivalent? Justify your answer in terms of what each one actually locks and when.
3. Explain, in your own words, why Prisma's tagged-template `$queryRaw` / `$executeRaw` are safe against SQL injection, while building the equivalent query with plain string concatenation would not be.

How to start

Don't start by trying to write working `$queryRaw` code. Start by answering research question 3 above on paper — understanding what `SELECT ... FOR UPDATE` is actually FOR is what makes question 1's sketch possible at all.

Hints

- **Hint 1 (conceptual):** `createIssueWithKey()` never reads a value, makes a decision based on it, and only then decides what to write — it just writes unconditionally. `SELECT ... FOR UPDATE` exists for the case where a decision happens BETWEEN the read and the write.
- **Hint 2 (where to look):** The study guide's "Why Postgres's row lock is enough here" reasoning (from the session's own reference material) directly answers research question 4 — re-read it before answering question 2 of the written CHALLENGE.
- **Hint 3 (API/tool name):** Prisma's raw SQL functions are tagged templates — `` `tx.$queryRaw` SELECT ... WHERE id = ${id} `` – every `${...}` interpolation becomes a separate real SQL parameter, never text pasted into the query string itself.

Rules

Allowed: everything already shown live or in teamwork — `$transaction`, `{ increment: 1 }`, `findUnique` with a compound-unique input, `$queryRaw` / `$executeRaw` (CHALLENGE research only, not required to implement).

Not yet: no new schema field, no new migration — this homework extends the existing mechanism's own proof, it does not add new structure.

Test it

BASH

```
# inside api/
npm install --save-exact --save-dev dotenv@16.4.5   # once, if not already installed
npx ts-node prisma/homework-concurrency-proof.ts
```

Both printed booleans (same-project serialized; different-project stayed parallel) must read `true`, from a REAL run — not asserted from reading the code.

Common confusions

- Treating `Environment variable not found: DATABASE_URL` as a sign something about the concurrency logic is wrong — it means the Setup step above (`import 'dotenv/config'`) was skipped; it has nothing to do with locking.
- Timing two SEQUENTIAL `await`ed calls instead of two calls started at the same time — sequential calls would trivially never overlap, proving nothing about locking at all.
- Placing the artificial delay on the wrong side of the operation that actually acquires the row lock — if nothing has been locked yet when the delay runs, the delay measures the wrong thing.
- Expecting the DIFFERENT-project timing to be exactly identical to a single call's time — a small real overhead is normal and not a failure.

Done when

- `prisma/homework-concurrency-proof.ts` exists and, when actually run, prints both expected results as `true`.
- The STRETCH comparison names all four real correspondences specifically.
- All three CHALLENGE answers name a concrete mechanism (a specific SQL sketch; a specific locking-scope justification; a specific parameterization explanation), not a restatement of the question or a generic "it's safer" answer.

Working time

About 25–35 minutes — the proof script closely follows the live-coded shape; the written answers deserve genuine thinking time.